

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
федеральное государственное автономное образовательное учреждение высшего образования
«САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
АЭРОКОСМИЧЕСКОГО ПРИБОРОСТРОЕНИЯ»

ИНСТИТУТ НЕПРЕРЫВНОГО И ДИСТАНЦИОННОГО ОБРАЗОВАНИЯ

КАФЕДРА №3

ОЦЕНКА

ПРЕПОДАВАТЕЛЬ

старший преподаватель
должность, уч. степень, звание

подпись, дата

Н.А. Соловьева
инициалы, фамилия

КОНТРОЛЬНАЯ РАБОТА
Вариант 14

по дисциплине: Технология программирования

РАБОТУ ВЫПОЛНИЛ

СТУДЕНТ гр. №

В3441
номер группы

подпись, дата

Р.С. Кулаков
инициалы, фамилия

Санкт-Петербург 2025

Вариант 14

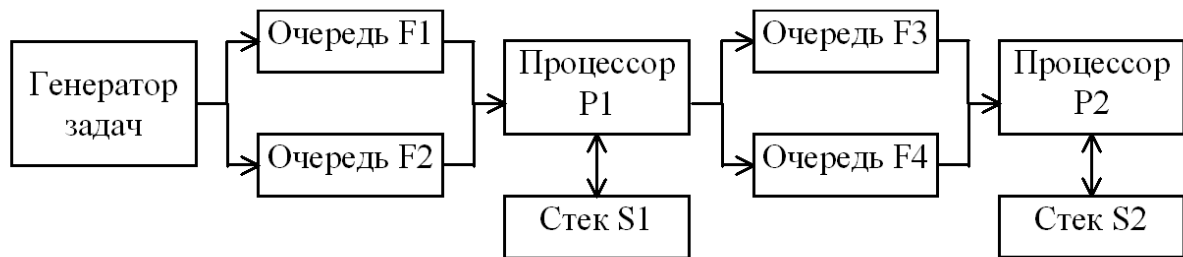
Кулаков Роман Станиславович	14	5	1	6	flow-форма
--------------------------------	----	---	---	---	------------

1 Задание 1

Для задачи по варианту составить таблицу решений на 5 – 6 условий. Если задача требует большего числа условий, то нужно ее переформулировать и упростить. Новую формулировку записать в ответ. Все комбинации условий (полный перебор всех комбинаций) должны быть представлены в таблице (5 условий – 32 комбинации, 6 условий – 64 комбинации). Количество действий должно быть таким, чтобы максимально отобразить содержание задачи.

Задача 5.

Система состоит из двух процессоров P1 и P2, двух стеков S1 и S2 и четырёх очередей F1, F2, F3, F4. В систему могут поступать запросы на выполнение задач двух приоритетов - высший (1) и низший (2). Задачи сначала обрабатываются последовательно процессором P1, затем P2.



Запросы на выполнение задач высшего приоритета, поступающие из генератора задач, ставятся в очередь F1, а поступающие с процессора P1 - в очередь F3. Запросы на выполнение задач низшего приоритета, поступающие с генератора задач, ставятся в очередь F2, а поступающие с процессора P1 - в очередь F4. Процессор P1 обрабатывает запросы из очередей F1 и F2, а процессор P2 - из очередей F3 и F4. Процессор сначала обрабатывает задачи из очереди задач с высшим приоритетом, затем из очереди задач с низшим приоритетом. Если

процессор выполняет задачу с низшим приоритетом и приходит запрос на выполнение задачи с высшим приоритетом, то выполняемая задача помещается в соответствующий процессору стек, а пришедшая задача - в процессор. Задача из стека возвращается в процессор, если все задачи большего приоритета обработаны.

1.1 Решение

В формате исходной постановки задача требует больше 5-6 условий, поэтому постановка упрощена.

1.2 Новая формулировка

Система состоит из одного процессора P , двух очередей Q_1 (высший приоритет) и Q_2 (низший приоритет) и одного стека S .

Процессор обрабатывает задачи из Q_1 , затем из Q_2 .

Когда поступает новая задача:

- если приоритет высокий $\rightarrow$ в Q_1 ,
- если низкий $\rightarrow$ в Q_2 .

Если процессор занят задачей низкого приоритета и приходит задача высокого приоритета, текущая низкоприоритетная задача прерывается и помещается в стек S , а задание высокого приоритета запускается сразу.

Когда Q_1 пуста и нет новых задач высокого приоритета, из стека S «вынимается» последняя прерванная задача и возобновляется её выполнение.

Ниже представлена расширенная таблица решений для 5 условий с подробным указанием действий. Вместо одной колонки «Действие» действия вынесены в отдельные колонки. В клетках стоит «+», где действие должно выполняться.

Возможные условия:

- $C1$ — Поступил новый запрос высокого приоритета? (Y/N)
- $C2$ — Поступил новый запрос низкого приоритета? (Y/N)
- $C3$ — Процессор свободен? (Y/N)
- $C4$ — В данный момент выполняется задача низкого приоритета? (Y/N)

- C5 — В очереди Q₁ нет задач? (Y/N)

Возможные действия:

- F1 — поставить в Q₁
- F2 — поставить в Q₂
- F3 — начать выполнение из Q₁
- F4 — начать выполнение из Q₂
- F5 — положить текущую задачу в стек S
- F6 — достать задачу из стека S и возобновить
- F7 — ничего не делать

№	C1	C2	C3	C4	C5	F1	F2	F3	F4	F5	F6	F7
1	N	N	Y	N	Y							+
2	N	N	Y	N	N						+	
3	N	N	Y	Y	Y							+
4	N	N	Y	Y	N						+	
5	N	N	N	N	Y							+
6	N	N	N	N	N							+
7	N	N	N	Y	Y							+
8	N	N	N	Y	N							+
9	N	Y	Y	N	Y		+					

10	N	Y	Y	N	N		+		+			
11	N	Y	Y	Y	Y		+					
12	N	Y	Y	Y	N		+					
13	N	Y	N	N	Y		+					
14	N	Y	N	N	N		+					
15	N	Y	N	Y	Y		+					
16	N	Y	N	Y	N		+					
17	Y	N	Y	N	Y	+		+				
18	Y	N	Y	N	N	+		+				
19	Y	N	Y	Y	Y	+		+		+		
20	Y	N	Y	Y	N	+		+		+		
21	Y	N	N	N	Y	+						
22	Y	N	N	N	N	+						
23	Y	N	N	Y	Y	+						
24	Y	N	N	Y	N	+						
25	Y	Y	Y	N	Y	+	+	+				
26	Y	Y	Y	N	N	+	+	+				
27	Y	Y	Y	Y	Y	+	+	+		+		

28	Y	Y	Y	Y	N	+	+	+		+		
29	Y	Y	N	N	Y	+	+					
30	Y	Y	N	N	N	+	+					
31	Y	Y	N	Y	Y	+	+					
32	Y	Y	N	Y	N	+	+					

2 Задание 2

1.. Дать пример таблицы с составным ключом. Таблица должна состоять минимум из 5 атрибутов. Объяснить, почему пример подходит под задание.

```
CREATE TABLE OrderDetails (
  OrderID INT,      -- Идентификатор заказа
  ProductID INT,    -- Идентификатор товара
  Quantity INT,     -- Количество единиц товара
  UnitPrice DECIMAL(10,2), -- Цена за единицу на момент заказа
  Discount DECIMAL(5,2), -- Скидка в процентах
  PRIMARY KEY (OrderID, ProductID)
);
```

1. Составной ключ:

- Вместо одного атрибута в качестве РК используются два — OrderID и ProductID.
- Ни OrderID (один заказ может содержать несколько товаров) ни ProductID (один товар может встречаться в разных заказах) по отдельности не

уникальны в этой таблице, но их комбинация гарантирует уникальность каждой строки.

2. Минимум 5 атрибутов:

Таблица содержит ровно 5 столбцов (OrderID, ProductID, Quantity, UnitPrice, Discount).

3 Задание 3

Для фрагмента программы нарисовать графическое представление алгоритма

Фрагмент 6, flow-форма

```
int main() {
    std::random_device rd;
    std::mt19937 gen(rd());
    std::uniform_int_distribution<> dis(0, 9);

    // Генерация 10 000 тестовых ключей
    Keys* GeneratedKeys = new Keys;
    Keys* Tmp = GeneratedKeys;
    for (int i = 0; i < NumOfKeys; i++) {
        Tmp->keys[0] = dis(gen);
        for (int j = 1; j < 6; j++) {
            Tmp->keys[j] = 'A' + rand() % 26;
        }
        Tmp->next = new Keys;
        Tmp = Tmp->next;
    }
    // Создание массива сегментов хеш-таблицы
    Keys* MassiveOfSegments = new Keys[NumOfSegments];
    for (int i = 0; i < NumOfSegments; i++) {
        for (int j = 0; j < 6; j++) {
            MassiveOfSegments[i].keys[j] = 0;
        }
        MassiveOfSegments[i].numOfH = 0;
        MassiveOfSegments[i].next = NULL;
    }

    Keys* BadKeys = NULL;

    // Вызов функции хеширования
    HashFunction(GeneratedKeys, MassiveOfSegments, BadKeys);
    // Сохранение результатов в файл
    std::ofstream outputFile("./results.txt");
```

```
if (outputFile.is_open()) {  
    for (int i = 0; i < NumOfSegments; i++) {  
        int numCollisions = MassiveOfSegments[i].numOfH;  
  
        outputFile << i << ": " << numCollisions << std::endl;  
    }  
}  
}
```

std::random_device rd;	
std::mt19937 gen(rd());	
std::uniform_int_distribution<> dis(0, 9);	
Keys* GeneratedKeys = new Keys;	
Keys* Tmp = GeneratedKeys;	
for i = 0 to NumOfKeys	
do	Tmp->keys[0] = dis(gen);
	for j = 0 to 6
	do
	Tmp->keys[j] = 'A' + rand() % 26;
	Tmp->next = new Keys;
Tmp = Tmp->next;	
Keys* MassiveOfSegments = new Keys[NumOfSegments];	
for i = 0 to NumOfSegments	
do	for j = 0 to 6
	do
	MassiveOfSegments[i].keys[j] = 0;
	MassiveOfSegments[i].numOfH = 0;
MassiveOfSegments[i].next = NULL;	
Keys* BadKeys = NULL;	
// Вызов функции хеширования	
HashFunction(GeneratedKeys, MassiveOfSegments, BadKeys);	
// Сохранение результатов в файл	
std::ofstream outputFile("../results.txt");	
if outputFile.is_open()	
then	for j = 0 to NumOfSegments
	do
	int numCollisions = MassiveOfSegments[i].numOfH;
outputFile << j << " " << numCollisions << std::endl;	